CSCI 2406: Topic 08

Procedures and the Stack

Procedure calls, return linkage, stack organization, parameter passing, and AAPCS64 calling conventions

Computer Organization & Architecture | Topic 8

Topic 8 Outline

Topic Objective

Examine the architectural mechanisms and software conventions that allow programs to execute subroutines, pass arguments, isolate local variables, and return control securely.

- **Section 1**: The Call and Return Mechanism (`BL`, `RET`)
- **Section 2**: The Hardware Stack and the Stack Pointer (`SP`)
- **Section 3**: The AAPCS64 Calling Convention
- **Section 4**: Practical C to Assembly Translation
- **Section 5**: Stack Frames and the Frame Pointer (`FP`)
- **Section 6**: Summary & Interactive Self-Test

Section 01

The Call and Return Mechanism

The Need for Procedures

Procedures (functions, subroutines, methods) are the primary abstraction for software modularity.

To execute a procedure, the hardware must successfully manage:

- **Control Transfer**: Jump to the subroutine's memory address.
- **State Preservation**: Ensure the caller's data is not destroyed by the callee.
- **Argument Passing**: Transmit inputs to the subroutine.
- **Value Return**: Transmit outputs back to the caller.
- **Return Linkage**: Remember exactly where to resume the original execution.

Branch and Link (BL)

The BL instruction solves the control transfer and return linkage problems simultaneously.

Execution semantics of `BL label`

1. Computes the return address: the address of the instruction immediately following the BL.
2. Stores this return address into the **Link Register (x30)**.
3. Overwrites the Program Counter (PC) with the address of `label`.

Why not just use B (Branch)?
A standard branch overwrites the PC but does *not* save the return address. The processor would have no way to find its way back to the caller.

Return (RET)

The RET instruction completes the subroutine lifecycle by transferring control back to the caller.

Execution semantics of RET

- Reads the memory address currently stored in x30 (the Link Register).
- Unconditionally branches to that address (PC $\leftarrow$ x30).

The Link Register Vulnerability

If a subroutine calls *another* subroutine (a nested call), the inner BL will overwrite x30. The original return address is lost forever unless it is backed up to memory before the nested call.

Section 02

The Hardware Stack

The Hardware Stack

To solve the nested call problem and provide space for local variables, architectures utilize a **Stack** in Main Memory (RAM).

- **Structure**: A Last-In, First-Out (LIFO) memory queue.
- **Direction**: The AArch64 stack is **Full Descending**. It grows from high memory addresses down toward lower memory addresses.
- **Stack Pointer (`SP`)**: A dedicated architectural register that holds the memory address of the last item pushed onto the stack.

Note: Unlike x86, AArch64 does not have dedicated `PUSH` and `POP` assembly instructions. We construct stack operations manually using load/store instructions.

Stack Pointer (SP) Alignment Constraints

The 16-Byte Rule

In AArch64, the Stack Pointer (SP) must **always be 16-byte aligned** when used as a base register for memory access.

- The memory address held in SP must end in 0x0 (i.e., be a multiple of 16).
- If you attempt a LDR or STR using an unaligned SP, the processor hardware generates an alignment fault (exception).
- Consequence: You must allocate stack space in chunks of at least 16 bytes, even if you only need to save an 8-byte register.

Simulating Push and Pop (STP / LDP)

We use **pre-indexed** and **post-indexed** addressing modes to manipulate the stack efficiently.

Pushing to the Stack

```
stp x29, x30, [sp, -16]!
```

1. Subtracts 16 from SP (growing the stack down).
2. Stores x29 and x30 into the newly allocated 16-byte slot.
3. The `!` updates SP to the new address permanently.

Popping from the Stack

```
ldp x29, x30, [sp], 16
```

1. Loads x29 and x30 from the current SP address.
2. Adds 16 to SP (shrinking the stack up).
3. SP is updated automatically *after* the load completes.

Section 03

The AAPCS64 Convention

The AAPCS64 Standard

The **ARM Architecture Procedure Call Standard** (AAPCS64) is the software contract that compilers (like GCC and Clang) follow. It dictates exactly which registers are used for arguments, returns, and variables.

Registers	Designated Purpose
`x0`–`x7`	Arguments 1 through 8 (also used to return values in `x0`–`x1`).
`x8`	Indirect result location register (struct returns).
`x9`–`x15`	Temporary registers (Caller-saved / Volatile).
`x19`–`x28`	Callee-saved registers (Non-volatile).
`x29` **(`FP`)**	Frame Pointer (anchors the local stack frame).
`x30` **(`LR`)**	Link Register (holds the return address).

Register Volatility: Caller-Saved vs. Callee-Saved

Caller-Saved (`x0`–`x18`)

- These registers are **volatile**.
- A subroutine is free to overwrite them without backing them up.
- If the caller cares about the data in `x0`–`x18`, the *caller* must push them to the stack before invoking `BL`.

Callee-Saved (`x19`–`x28`)

- These registers are **non-volatile**.
- If a subroutine wants to use them, the *callee* must back them up to the stack first, and restore them before calling `RET`.
- The caller trusts these will not change across a function call.

Section 04

Practical C to Assembly

C to Assembly Translation: Procedure Calls

C Source Code

```
long sum(long a, long b) {
    return a + b;
}

long main() {
    return sum(5, 10);
}
```

Compiler behavior:

- Maps a to x0, b to x1.
- Expects return value in x0.
- Must preserve x30 in main because

AArch64 Assembly (GCC -O1)

```
sum:
    add     x0, x0, x1
    ret

main:
    // 1. Prologue: Save FP and LR
    stp     x29, x30, [sp, -16]!
    mov     x29, sp

    // 2. Setup Arguments
    mov     x1, 10
    mov     x0, 5
```

Deconstructing the Compilation Strategy

- **The Leaf Function (`sum`)**: Because `sum` does not call any other function, it is a "leaf." It does not overwrite `x30`, so it does not need to allocate stack space. It simply performs math and returns immediately.
- **The Nested Function (`main`)**: `main` is called by the OS loader (meaning `x30` contains the OS return address). Because `main` calls `sum`, the `bl sum` instruction will destroy the OS return address.
- **The Solution**: `main` pushes `x30` (and `x29`) to the stack in its **Prologue**, executes the program, and pops them back in its **Epilogue** before returning.

Section 05

Stack Frames

Stack Frames (Activation Records)

A **Stack Frame** is the block of memory allocated on the stack for a single execution of a subroutine.

What goes in a Stack Frame?

- Saved registers (like `x30` LR and `x29` FP).
- Saved Callee-saved registers (`x19`–`x28`) if the function modifies them.
- Local variables (arrays, structs, or variables that do not fit in registers).
- Spilled arguments (if a function takes more than 8 parameters, arguments 9+ are passed on the stack).

The Frame Pointer (x29)

As functions push and pop variables, the Stack Pointer (SP) moves continuously. This makes addressing local variables via `[sp, #offset]` difficult, as the offset changes dynamically.

- **The Frame Pointer (x29 / FP)**: Serves as a fixed reference anchor for the current stack frame.
- Standard practice immediately after pushing x29/x30 is to copy the stack pointer: `mov x29, sp`.
- Compilers map local variables relative to x29 (e.g., `[x29, #16]`). This address remains completely stable regardless of how much additional data is pushed to SP.
- FP also forms a linked list of frames, allowing debuggers to trace the call stack in the event of a crash.

Section 06

Summary & Self-Test

Topic 8 Summary

Concept	Architectural Takeaway
`BL` and `RET`	`BL` sets `PC` and saves the return address to `x30`. `RET` jumps back to `x30`.
The Hardware Stack	Full descending queue in RAM. `SP` must always remain strictly 16-byte aligned.
AAPCS64 Standard	`x0–x7` for args. `x9–x15` are volatile. `x19–x28` are non-volatile (callee-saved).
Prologue / Epilogue	Functions calling other functions must back up `x30` and `x29` to the stack immediately.
Frame Pointer	`x29` provides a stable memory anchor for addressing local variables dynamically.

Self-Test: Concept Check 1

Question 1: Subroutine Calls

You are analyzing AArch64 assembly and notice a subroutine that does **not** push the Link Register (`x30`) to the stack, yet the program executes and returns successfully.

Why is this possible?

A) The subroutine is a "leaf" function that makes no inner `BL` calls.

B) The hardware automatically backs up `x30` to the stack.

C) The compiler used a caller-saved register to store the return address instead.

D) The code violates AAPCS64 but gets lucky at runtime.

Self-Test: Concept Check 1

Question 1: Subroutine Calls

You are analyzing AArch64 assembly and notice a subroutine that does **not** push the Link Register (`x30`) to the stack, yet the program executes and returns successfully.

Why is this possible?

A) The subroutine is a "leaf" function that makes no inner `BL` calls.

B) The hardware automatically backs up `x30` to the stack.

C) The compiler used a caller-saved register to store the return address instead.

D) The code violates AAPCS64 but gets lucky at runtime.

Answer: A

If a function never executes a `BL` instruction, `x30` is never overwritten, so it does not need to be backed up to the stack.

Self-Test: Concept Check 2

Question 2: Stack Management

An AArch64 developer writes the following instruction to back up the `x19` register before modifying it: `str x19, [sp, -8]!`

What will happen when this code runs on physical hardware?

A) `x19` is successfully pushed to the stack.

B) The assembler will convert it to a 16-byte push automatically.

C) The processor will trigger a hardware alignment fault and crash.

D) The data is stored, but `SP` is not updated.

Self-Test: Concept Check 2

Question 2: Stack Management

An AArch64 developer writes the following instruction to back up the `x19` register before modifying it: `str x19, [sp, -8]!`

What will happen when this code runs on physical hardware?

A) `x19` is successfully pushed to the stack.

B) The assembler will convert it to a 16-byte push automatically.

C) The processor will trigger a hardware alignment fault and crash.

D) The data is stored, but `SP` is not updated.

Answer: C

The AArch64 Stack Pointer (`SP`) must be strictly 16-byte aligned. Subtracting 8 from `SP` breaks alignment and causes an immediate hardware exception on memory access.

Wrap-Up & Next Steps

What We Achieved

We successfully bridged the gap between high-level C functions and hardware-level procedure mechanics. You can now trace execution flow across modular program architectures, manage the stack securely, and respect AAPCS64 software contracts.

Looking Ahead to Topic 9

With control flow and data representation mastered, we will turn our attention to **Machine Instruction Encoding**. We will examine the raw binary fields that make up 32-bit AArch64 instructions to understand exactly how the hardware decodes the assembly syntax we have been writing.